Web and PDF Research Sub-Agent: Design

Stack: LangGraph, LangChain, OpenAI web search, PostgreSQL (pgvector), BeautifulSoup, OpenShift CronJobs
Role: One sub-agent behind the router, alongside the Text-to-SQL agent and the internal RAG agent.

---

1. Purpose

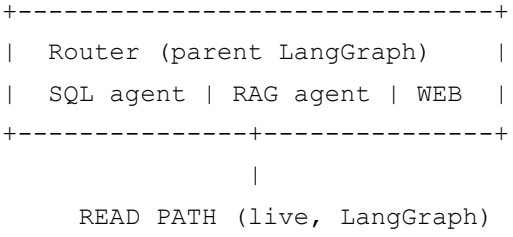
Answer questions about Singtel, its subsidiaries, and market peers (Airtel and others) using public web sources. The agent must support three capabilities:

#	Capability	When it runs	Output
A	**PDF ingestion** (batch)	Scheduled	PDFs downloaded, extracted, chunked, embedded
B	**Scheduled web search and page crawl** (batch)	Scheduled	Web pages and search findings stored and embedded
C	**Live web search**	On user question	Fresh answer grounded in live results

Core principle: split into a **write path** (batch jobs, no LLM agent loop) and a **read path** (a small LangGraph sub-agent). Users get fast answers from the store, and a broken website never breaks an answer.

---

2. Architecture



```
          classify -> retrieve -> grade -> [live_search] -> answer
                                  |
                        +---------v---------+
                        |   PostgreSQL + pgvector       |
                        |  sources | search_jobs        |
                        |  documents | chunks           |
                        |  ingest_runs | page_cache     |
                        +---------^---------+
                                  |
                             WRITE PATH (OpenShift CronJobs)
                      +-----------------+-----+--------------------+
                      | Job A: PDF      |  Job B: page crawl +     |
                      | ingestion       |  scheduled web search    |
                      +-----------------+--------------------------+
                                  |
                            Egress proxy (allowlisted domains)
                                  |
                       Singtel / Airtel / subsidiary sites, OpenAI API
```

---

3. Data Model (PostgreSQL)

```
CREATE EXTENSION IF NOT EXISTS vector;

-- Config: known websites and how to read them
CREATE TABLE sources (
  id              SERIAL PRIMARY KEY,
  entity          TEXT NOT NULL,            -- Singtel, Optus, NCS, Airtel...
  url             TEXT NOT NULL,
  kind            TEXT NOT NULL,            -- 'page' | 'pdf_index'
  allowed_domain  TEXT NOT NULL,
  pdf_selector    TEXT,                     -- CSS selector for PDF links (pdf_in
  text_selector   TEXT,                     -- optional CSS selector for main con
  doc_type        TEXT,                     -- annual_report | results | presenta
  needs_js        BOOLEAN DEFAULT false,
  refresh_hours   INT DEFAULT 24,
  active          BOOLEAN DEFAULT true,
  last_fetched_at TIMESTAMPTZ
);

-- Config: scheduled web searches (Capability B)
CREATE TABLE search_jobs (
  id              SERIAL PRIMARY KEY,
  entity          TEXT NOT NULL,
```

```
  query           TEXT NOT NULL,            -- e.g. 'Airtel Africa latest quarter
  allowed_domains TEXT[],                   -- scope for OpenAI web_search
  refresh_hours   INT DEFAULT 24,
  active          BOOLEAN DEFAULT true,
  last_run_at     TIMESTAMPTZ
);

-- One row per fetched document (PDF, page, or search finding)
CREATE TABLE documents (
  id            SERIAL PRIMARY KEY,
  source_id     INT REFERENCES sources(id),
  search_job_id INT REFERENCES search_jobs(id),
  entity        TEXT NOT NULL,
  origin        TEXT NOT NULL,              -- 'pdf' | 'page' | 'batch_search' |
  url           TEXT NOT NULL,
  url_hash      TEXT NOT NULL,
  content_hash  TEXT,                       -- skip re-embedding if unchanged
  title         TEXT,
  period        TEXT,                       -- FY25, Q2 FY26 (where known)
  status        TEXT DEFAULT 'discovered',  -- discovered | downloaded | indexed
  retry_count   INT DEFAULT 0,
  last_error    TEXT,
  fetched_at    TIMESTAMPTZ,
  expires_at    TIMESTAMPTZ,                -- NULL for PDFs, short TTL for pages
  UNIQUE (url_hash)
);

CREATE TABLE chunks (
  id          BIGSERIAL PRIMARY KEY,
  document_id INT REFERENCES documents(id) ON DELETE CASCADE,
  entity      TEXT NOT NULL,
  origin      TEXT NOT NULL,
  page        INT,                          -- PDF page; NULL for web
  text        TEXT NOT NULL,
  embedding   VECTOR(1536),                 -- match your embedding model
  fetched_at  TIMESTAMPTZ,
  expires_at  TIMESTAMPTZ
);
CREATE INDEX ON chunks USING hnsw (embedding vector_cosine_ops);
CREATE INDEX ON chunks (entity, origin);

-- Health log for every batch run
CREATE TABLE ingest_runs (
  id          BIGSERIAL PRIMARY KEY,
  job_name    TEXT,
  source_id   INT,
```

```
    status      TEXT,                          -- ok | partial | failed
    items_found INT,
    items_new   INT,
    error       TEXT,
    started_at  TIMESTAMPTZ,
    ended_at    TIMESTAMPTZ
);
```

Postgres doubles as the job queue: workers pick rows with `status IN ('discovered','failed') AND retry_count < 3` using `FOR UPDATE SKIP LOCKED`. No separate queue system is needed at this scale.

---

4. Write Path: Batch Jobs (OpenShift CronJobs)

All jobs are plain Python, with no agent loop. They are idempotent and safe to re-run.

Job A: PDF ingestion (daily)

1. For each active `pdf_index` source: fetch the page (requests, or Playwright if `needs_js`).
2. Parse with **BeautifulSoup** and apply `pdf_selector` to collect PDF links. Resolve relative URLs.
3. Insert new URLs into `documents` (`origin='pdf'`, `status='discovered'`). Existing `url_hash` values are skipped.
4. Worker loop over `discovered` rows:
   - Download with a size cap (20 MB) and a content-type check.
   - Extract text per page with **PyMuPDF**, and tables with **pdfplumber**. Keep tables as markdown and record units and currency.
   - OCR (Tesseract) only if a page has no extractable text.
   - Chunk by page and section (about 800 tokens, 100 overlap), embed, write `chunks`.
   - In one transaction, replace the document's chunks and set `status='indexed'`.
5. Zero PDF links found on a page means the site layout changed. Log it in `ingest_runs` and alert.

Job B1: Page crawl (daily or hourly, per `refresh_hours`)

1. For each active `page` source that is due: fetch, then extract clean text (BeautifulSoup with `text_selector`, or **trafilatura** as the default).

2. Compute `content_hash`. If unchanged, only update `fetched_at`. If changed, replace chunks and re-embed.
3. While parsing, harvest any PDF links and insert them as `discovered` for Job A.

Job B2: Scheduled web search (daily, per `refresh_hours`)

1. For each due `search_jobs` row, call OpenAI web search via the Responses API, scoped to `allowed_domains`.
2. Store the returned answer text as a `batch_search` document, with its cited URLs as metadata.
3. Fetch each cited URL (BeautifulSoup or trafilatura) and store it as a `page` document tagged with the `search_job_id`. This keeps the original source text, not just the model's summary.
4. Chunk, embed, write with `expires_at` (for example 7 days).

> Check the current OpenAI docs for the exact domain-filter parameter on the web search tool (`allowed_domains`) and for response citation fields, as these have changed between releases.

Operational rules for all jobs

- Per-domain delay of 1-2 seconds, a fixed User-Agent, and respect `robots.txt`.
- Timeouts on every request, retries with backoff (**tenacity**), max 3 attempts, then `failed`.
- One source failing never stops the others. Wrap each source in its own try/except and log to `ingest_runs`.
- Concurrency of 1 CronJob instance per job (`concurrencyPolicy: Forbid`).

---

5. Read Path: LangGraph Sub-Agent (Capability C)

A small graph with one conditional edge and no loops.

```
START -> classify -> retrieve -> grade --(sufficient)--> answer -> END
                                    |
                                    +--(insufficient / needs latest)--> live_searc
```

Nodes

Node	What it does
classify	Maps the question to entities (and period if mentioned) using the `sources` table. Flags whether the question asks for "latest/current/today" information.
retrieve	pgvector search filtered by `entity`, excluding expired chunks (`expires_at IS NULL OR expires_at > now()`). Top-k of 6 to 8.
grade	Simple rule first: top similarity score below threshold, no chunks, or the question asks for "latest" and the newest chunk is older than its TTL. Optionally one cheap LLM call to judge sufficiency.
live_search	OpenAI web search scoped to the entity's `allowed_domains`. Optionally fetches the top 2-3 cited pages with BeautifulSoup for fuller text. Results are held in state and, optionally, written back as `live_search` documents with a short TTL (hours).
answer	LLM answers only from the evidence. Citations: URL plus page (PDFs) or URL plus "as of" date (web). Marks stored versus live evidence.

State

```
class WebState(TypedDict):
    question: str
    entities: list[str]
    period: str | None
    wants_latest: bool
    chunks: list[dict]        # from the store
    live_results: list[dict]  # from live search
    answer: str
    sources: list[dict]       # {type, url, page, as_of}
    confidence: float
    error: str | None
```

Wiring

```
g = StateGraph(WebState)
g.add_node("classify", classify)
g.add_node("retrieve", retrieve)
g.add_node("grade", grade)
g.add_node("live_search", live_search)
g.add_node("answer", answer)
```

```
g.add_edge(START, "classify")
g.add_edge("classify", "retrieve")
g.add_edge("retrieve", "grade")
g.add_conditional_edges("grade", lambda s: "live" if s["need_live"] else "done",
                        {"live": "live_search", "done": "answer"})
g.add_edge("live_search", "answer")
g.add_edge("answer", END)
web_agent = g.compile()
```

For multi-entity questions (for example "compare Singtel and Airtel"), fan out one branch per entity with LangGraph's `Send` API, then merge in the parent synthesizer. Add this only when needed.

---

6. Router Integration

- The parent router calls `web_agent` as a node or tool.
- **Route to Web when:** the question concerns public company or market information, external reports, or "latest" news.
- **Route to RAG when:** the question concerns internal documents.
- **Route to SQL when:** the question concerns internal structured data.
- Mixed questions fan out in parallel to several agents, then a synthesizer merges the results.

Common output contract (all agents):

```
{
  "answer": "...",
  "sources": [{"type": "pdf|page|live", "ref": "url", "page": 12, "as_of": "2026-
  "confidence": 0.82,
  "error": null
}
```

---

7. Security and Guardrails

- **Domain allowlist:** only domains in `sources` and `search_jobs` are ever fetched. Enforce in code and at the egress proxy.
- **SSRF protection:** resolve the host and block private and link-local IP ranges before fetching.

- **Prompt injection:** all page and PDF text is untrusted data. It is never placed in system instructions, and it can never trigger tools or SQL.
- **Limits:** 20 MB per PDF, 100 pages per PDF, 5 pages per live fetch, timeouts on every call, and a hard cap of 1 live search per question.
- **Least privilege:** the web agent's DB role can read `chunks` and `sources`. Only the batch jobs can write.
- **Secrets:** OpenAI API key and DB credentials in OpenShift Secrets.
- **Data egress (important for on-prem):** web search queries and any text sent to OpenAI for embeddings or answering leave your network. Never include internal data in web queries. Consider local embeddings (for example BGE-M3 or E5 served on-prem) if policy requires it.

---

8. Freshness Strategy

Content	TTL	Refresh
PDFs (reports, results)	None (permanent)	New PDFs discovered daily
Static pages (about, profile)	7 days	Daily crawl, hash check
News and announcements pages	1 day	Hourly or daily crawl
Batch search findings	7 days	Per `search_jobs.refresh_hours`
Live search write-back	6-24 hours	Expires automatically

A nightly cleanup job deletes expired rows. If a refresh fails, answers use the last good copy and show its date.

---

9. OpenShift Deployment

Component	Type	Notes
`web-agent`	Deployment (2 replicas)	LangGraph sub-agent API

`job-pdf-ingest`	CronJob (daily, 02:00)	`concurrencyPolicy: Forbid`
`job-page-crawl`	CronJob (hourly)	Only processes sources that are due
`job-batch-search`	CronJob (daily, 03:00)	Runs due `search_jobs`
`job-cleanup`	CronJob (daily)	Deletes expired chunks and documents
PostgreSQL	StatefulSet or managed service	pgvector enabled; also hosts the LangGraph checkpointer
Egress	Proxy plus NetworkPolicy	Only allowlisted domains and `api.openai.com`

- One container image for the jobs and the agent, with different entrypoints.
- Playwright needs a browser image variant. Use it only for sources with `needs_js = true`.
- Resource limits on the jobs, especially the PDF job (memory for large files).

---

10. Observability

- **`ingest_runs` dashboard:** per-source status, last success, items found and new. Alert when a source has zero links or three consecutive failures.
- **LangGraph tracing:** LangSmith, or a self-hosted alternative such as Langfuse if you cannot send traces off-prem.
- **Per-request log:** which entities were detected, whether the live path was used, top retrieval scores, and latency.
- **Evaluation set:** 30-50 labeled questions (PDF-answerable, page-answerable, needs-live) to track retrieval quality and the grade-to-live decision.

---

11. Suggested Additions to Your Stack

Addition	Why
trafilatura	Cleaner main-text extraction than BeautifulSoup alone; use

	BeautifulSoup for link and selector work
PyMuPDF + pdfplumber	Fast text extraction plus reliable table extraction
Tesseract (OCR)	Fallback for scanned PDFs only
Playwright	JS-rendered pages (flag per source)
tenacity	Retries with backoff
langchain-postgres (PGVector)	Ready-made vector store integration
langgraph-checkpoint-postgres	Resumable runs and debugging, in the same database
Local embedding model	Keeps document text on-prem if required
Langfuse	On-prem tracing option

---

12. Build Order

1. **Schema and seed config:** create tables, add 5-10 real `sources` rows (Singtel, Airtel, one subsidiary).
2. **Job A (PDF ingestion):** the highest-value path; verify table extraction on a real annual report.
3. **Read path v1:** classify, retrieve, answer (no live search yet). Test with the evaluation set.
4. **Job B1 (page crawl):** add page content and PDF link harvesting.
5. **Live search node:** add grade and live_search with the conditional edge.
6. **Job B2 (scheduled search):** add `search_jobs` for market-data queries.
7. **Hardening:** egress proxy, alerts, cleanup job, router integration.

---

13. Open Questions

1. Is sending queries and text to OpenAI (search, embeddings, answering) acceptable under your on-prem policy, or do embeddings and answering need to be local?
2. Which document types matter most (annual reports, quarterly results, presentations)?

This drives chunking and table handling.

3. Roughly how many sources and `search_jobs` do you expect at launch (tens or hundreds)?
4. Should live search results be written back to the store, or stay ephemeral?
5. Do any target sites require login, block bots, or restrict scraping in their terms?